Docker & Docker Compose

An Introductory Guide for Secure Computing Labs

Speaker: **Gregorio Toscano**

Subject: **Introduction to Containers for Security Experiments**

🐳 **Docker** | 🔧 **Docker Compose**

Why Containers in Secure Computing? 🔒

- **Purpose:** Use containers to run various software environments safely.
- **Experiments:** Deploy tools like **Kali Linux** or intentionally vulnerable apps.
- **Benefits:**
  - **Isolation:** Containers keep experiments separate from the host system.
  - **Flexibility:** Easily switch environments without hardware changes.
  - **Repeatability:** Create consistent, reproducible labs.

Using Docker, we can simulate real-world scenarios while keeping our systems secure.

What is Docker? 🐳

- **Docker** packages applications and their dependencies into **containers**.
- **Key features:**
  - **Lightweight:** Shares the host OS kernel.
  - **Isolated:** Own file system, CPU, memory, and process space.
- **Advantages:**
  - **Consistency:** “It works on my machine” → works everywhere.
  - **Efficiency:** Faster startup than virtual machines.
  - **Portability:** Build once, run anywhere.

Core Docker Components ⚙️

- **Docker Engine (daemon):** Runs on the host to manage images and containers.
- **Docker Client (CLI):** Command-line tool (`docker run ...`) that interacts with the daemon.
- **Docker Images:** Read-only templates used to create containers.
- **Docker Containers:** Running instances of images; can be ephemeral or persistent.

Basic Docker Commands 💻

```
# 1. Check Docker is running
docker version

# 2. Pull an image from Docker Hub
docker pull hello-world

# 3. Run a container
docker run hello-world

# 4. List running containers
docker ps
```

More Docker Commands 💻

```
# 5. List all containers (stopped + running)
docker ps -a

# 6. Remove a container
docker rm <container-id>

# 7. Remove an image
docker rmi <image-id>
```

Dockerfile: Building Your Own Image 🛠️

```dockerfile
FROM python:3.10-bullseye
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV DEBIAN_FRONTEND=noninteractive
ENV LANG="en_US.UTF-8"
ENV LC_ALL="en_US.UTF-8"
ENV LC_CTYPE="en_US.UTF-8"
RUN ln -fs /usr/share/zoneinfo/America/New_York /etc/localtime
RUN apt-get update && apt-get upgrade -y
RUN apt-get -y install apt-utils locales-all locales vim
RUN locale-gen en_US.UTF-8
RUN dpkg-reconfigure --frontend noninteractive tzdata
RUN apt-get install -y build-essential \
  python3-pip python-is-python3 python3-psycopg2
RUN pip install --upgrade pip && pip install pipenv
WORKDIR /app/
COPY --chown=www-data:www-data . ./
RUN pipenv install --system --deploy
USER www-data
EXPOSE 8000
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Building and Running an Image 🏗️

```
# Build an image from your Dockerfile
docker build -t my-django-app .

# Run a container from your new image
docker run -p 8000:8000 --name my-container my-django-app
```

- **Tag:** `-t my-django-app` labels your image.
- **Port Mapping:** `-p 8000:8000` exposes container port 8000.
- **Naming:** `--name my-container` assigns a name to the container.

Docker Compose 🚀

What is Docker Compose? 📄

- **Docker Compose** manages multi-container applications.
- **Modern usage:** Now integrated as a parameter (`docker compose up`).
- **Configuration:** Uses a **YAML file** to define services, networks, volumes, etc.
- **Simplicity:** One command starts all defined services.

Example Docker Compose File 📝

```yaml
services:
  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - .:/app
    ports:
      - "8000:8000"
    depends_on:
      - db

  db:
    image: postgres:13-alpine
    environment:
      POSTGRES_DB: mydatabase
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
    ports:
      - "5432:5432"
```

- **Services:** Defines `web` (the app) and `db` (PostgreSQL).
- **Build:** Uses a local Dockerfile for the `web` service.
- **Volumes:** Mount project directory for live updates.
- **Dependencies:** `depends_on` ensures the proper startup order.

Running Docker Compose 🏃‍♀️

1. **Create** a `docker-compose.yml` file in your project directory.
2. **Build and start** the application:

   ```
   docker compose up -d
   ```

   - `-d` runs containers in detached mode.
3. **View logs:**

   ```
   docker compose logs -f
   ```

4. **Stop the containers:**

   ```
   docker compose stop
   ```

5. **Clean up:**

   ```
   docker compose down
   ```

Additional Compose Features 🔍

- **Networks:** Automatically creates a default network for container communication.
- **Volumes:** Persist data and share it among containers.

```yaml
volumes:
  - db-data:/var/lib/postgresql/data
```

- **Environment Variables:** Manage via `.env` files or inline definitions.
- **Scaling:** Easily scale services (e.g., `docker compose up --scale web=3`).

Containers for Security Labs 🔐

- **Use Cases:**
  - **Kali Linux:** Run penetration testing tools safely.
  - **Vulnerable Software:** Deploy controlled environments to practice exploit techniques.
- **Isolation:** Containers segregate experiments, protecting your host system.
- **Flexibility & Reproducibility:** Quickly set up, tear down, and repeat experiments.

Security Considerations with Containers 🔒

- **Isolation vs. Kernel Sharing:** Containers isolate processes but share the host kernel.
- **Best Practices:**
  - Limit container privileges.
  - Regularly update images and dependencies.
  - Leverage Docker security features like user namespaces and seccomp profiles.
- **Reminder:** Containers improve lab safety, but configuration is key to security.

Demo Time: Spinning Up Kali Linux 🐧

Let's Launch a Kali Linux Container with Docker Compose!

In a new directory, create a `docker-compose.yml` with the following:

```yaml
services:
  kali-linux:
    image: lscr.io/linuxserver/kali-linux:latest
    container_name: kali-linux
    security_opt:
      - seccomp:unconfined # optional for certain tests
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=America/New_York
      - SUBFOLDER=/ # optional
      - "TITLE=Kali Linux" # optional
    volumes:
      - ./kali-data:/config
      - /var/run/docker.sock:/var/run/docker.sock # optional
    ports:
      - 3000:3000
      - 3001:3001
    devices:
      - /dev/dri:/dev/dri # optional
    shm_size: "2gb" # optional
    restart: unless-stopped
```

Launching Kali Linux 🐧

```
docker compose up -d
```

Then open your browser and navigate to `http://localhost:3000` to access the Kali Linux environment. This setup allows you to safely experiment with security tools and vulnerable applications.

Recap & Q&A 🤔

- **Docker:** Packages and isolates applications.
- **Docker Compose:** Simplifies multi-container orchestration.
- **Containers in Labs:** Enable secure, repeatable experiments.
- **Security Reminder:** Always configure containers with best practices.

Thank You! 🙏

Questions?

📧 **Email:** gtoscano@cua.edu

🌍 **Website:** cua.edu, Homepage, LinkedIn

🏢 **Office:** Pangborn Hall 309, CS/Engineering/The Catholic University of America